МИНОБРНАУКИ РОССИИ

Санкт-Петербургский государственный

электротехнический университет

«ЛЭТИ» им. В.И. Ульянова (Ленина)

Кафедра САПР

Практическая работа №2

по дисциплине

«Объектно-ориентированное программирование»

Студент гр. 5352 _____ Соболевский М.А.

Преподаватель _____ Кулагин М.В.

Санкт-Петербург

2026

1 Задание

Разработать консольное приложение на языке C, взаимодействующее с внешним HTTP-API по протоколу HTTPS. Приложение должно:

- реализовывать асинхронное (неблокирующее) взаимодействие с сетевыми ресурсами, не блокируя пользовательский интерфейс на время выполнения запроса;
- поддерживать интерактивный интерфейс пользователя (REPL-цикл), позволяющий отправлять запросы и получать результаты без перезапуска программы;
- корректно обрабатывать сетевые ошибки: недоступность сервера, ошибки TLS-рукопожатия, ошибки парсинга HTTP и JSON, таймауты;
- отображать прогресс выполнения длительных операций (индикатор активности, не засоряющий вывод);
- обеспечивать чистое разделение ответственности между слоями приложения: транспортом, клиентом API и пользовательским интерфейсом;
- демонстрировать применение принципов объектно-ориентированного проектирования в условиях языка C: инкапсуляция состояния, композиция компонентов, полиморфизм через интерфейсы, абстракция.

В качестве целевого API выбран поисковый API Википедии: программа принимает от пользователя поисковый запрос, выполняет HTTPS-запрос к `ru.wikipedia.org`, отображает список найденных статей и по выбору пользователя открывает соответствующую статью в браузере системы.

2 Спецификация программы

Архитектура программы представлена на рисунке 1. Приложение разделено на четыре слоя, каждый из которых знает только о соседнем нижнем слое и не имеет обратных зависимостей.

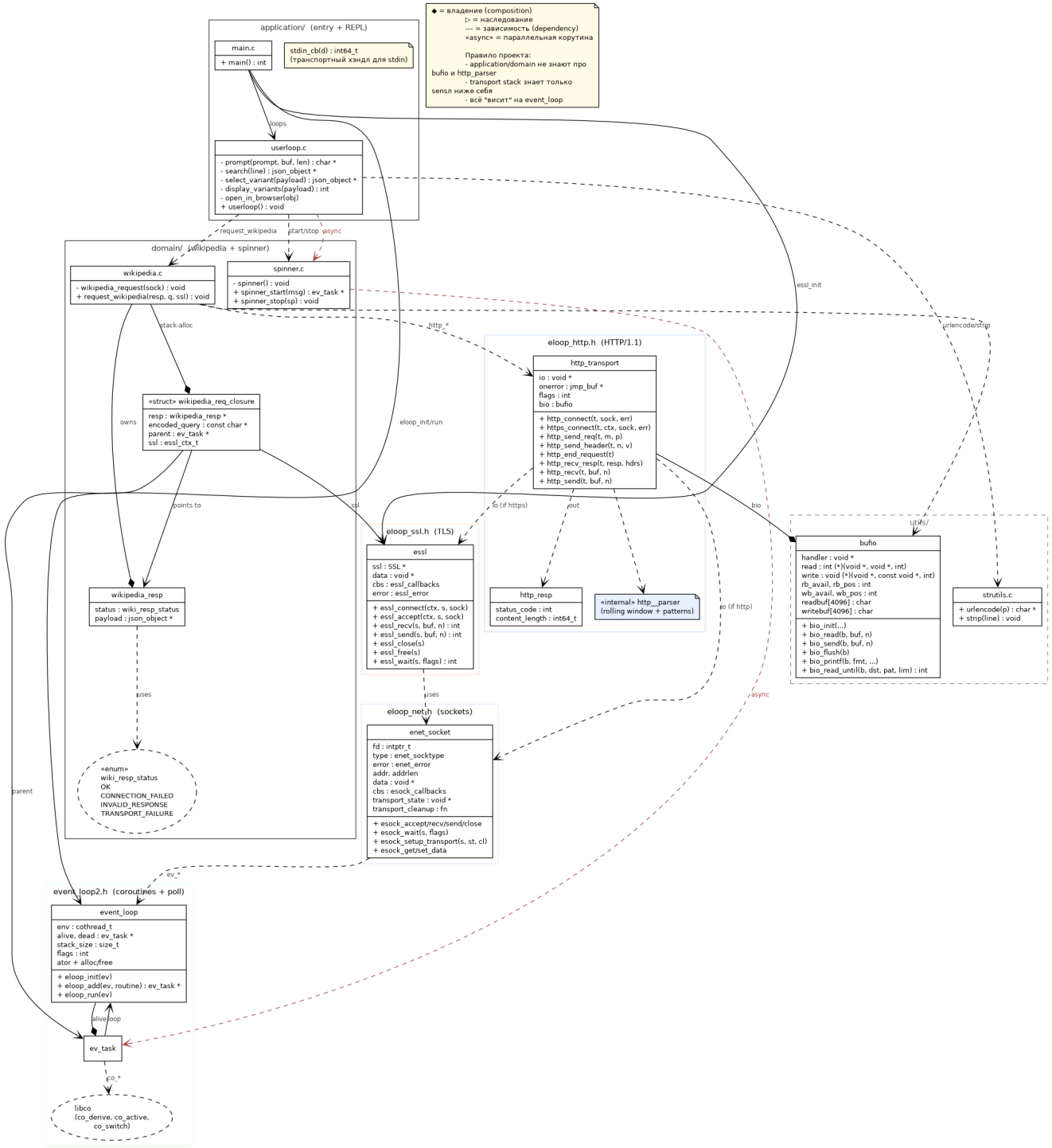


Рис. 1: Диаграмма компонентов

Жизненный цикл асинхронного запроса показан на рисунке 2. Каждый сетевой запрос выполняется в отдельной сопрограмме, которая приостанавливается на каждой потенциально блокирующей операции и возобновляется event-loop'ом при готовности сокета.

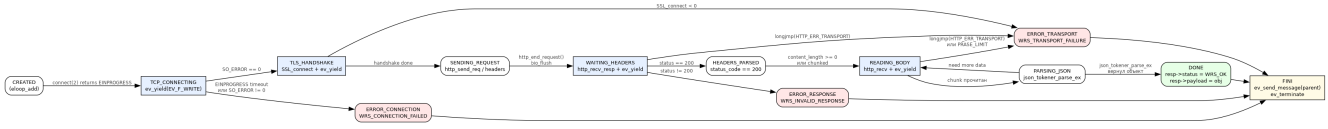


Рис. 2: Состояния сопрограммы сетевого запроса

Слои приложения

1. **Транспортный слой** (`event_loop2`, `eloop_net`, `eloop_ssl`, `eloop_http`) — внешняя библиотека, построенная на сопрограммах поверх `libco` и системного вызова `poll`. Обеспечивает неблокирующие сокеты, TLS-соединение (через OpenSSL) и HTTP-клиент с инкрементальным парсером заголовков. Слой не знает, как используется: приложение вызывает функции вида `http_send_req`, `http_recv_resp`, и получает данные по мере их поступления.

2. **Уровень клиента API** (`wikipedia.c`) — один метод `request_wikipedia`, который выполняет полный цикл: соединение, отправку HTTP-запроса, чтение тела ответа, парсинг JSON и возврат результата вызывающему. Всё взаимодействие с сетью инкапсулировано внутри; вызывающий видит только `struct wikipedia_resp`.

3. **Уровень пользовательского интерфейса** (`userloop.c`, `spinner.c`) — REPL-цикл, индикатор прогресса, отображение результатов и выбор пользователя. Не знает ни о сокетах, ни об HTTP, ни об SSL; работает только с `struct wikipedia_resp`.

4. **Вспомогательный слой** (`io/bufio.c`, `strutils.c`) — буферизация ввода-вывода и утилиты (URL-кодирование, отсечение пробелов).

2.1 Общие сведения

Программа представляет собой интерактивный консольный клиент поиска по Википедии. Взаимодействие построено по схеме REPL (read–eval–print loop): программа выводит приглашение `>`, читает строку со стандартного ввода, выполняет поиск и снова возвращается к приглашению.

Программа не принимает аргументов командной строки. Все действия выполняются через интерактивный ввод. Завершение программы идет по сигналу EOF (комбинация `Ctrl+D`) или принудительно `Ctrl+C`. Никаких файлов на диске программа не создаёт и не читает.

2.2 Запуск и завершение

Запуск:

```
$ ./wikipedia-cli
> _
```

После запуска программа выводит приглашение и ожидает ввода поискового запроса. Ввод завершается нажатием `Enter`. Пустая строка игнорируется только на уровне поиска (Википедия вернёт пустой результат).

Завершение работы с освобождением ресурсов происходит с комбинацией `Ctrl+D` (EOF на стандартном вводе). Программа корректно закрывает SSL-контекст, освобождает память и выходит с кодом 0.

2.3 Сценарий поиска

Поиск выполняется в четыре шага.

Шаг 1. Ввод запроса. Пользователь вводит запрос после приглашения:

```
> functional programming
```

Шаг 2. Выполнение запроса. Пока идёт HTTPS-соединение и сервер формирует ответ, программа отображает вращающийся индикатор активности — символы `-`, `\`, `|`, `/`, сменяющие друг друга с периодом 100 мс. Индикатор рисуется в одной позиции строки с помощью символа возврата каретки `\r`, поэтому не засоряет вывод:

```
> functional programming
Searching... -
```

Шаг 3. Вывод результатов. По завершении запроса индикатор стирается, и программа выводит пронумерованный список найденных статей:

```
1: Функциональное программирование
2: Лямбдаисчисление-
3: Haskell
4: OCaml
5: Функтор
6: Монада программирование()
7: Чистота программирование()
8: Ленивые вычисления
9: Комбинаторная логика
10: Схема язык( программирования)
```

Википедия по умолчанию возвращает 10 результатов на запрос.

Шаг 4. Выбор страницы. Программа выводит приглашение `select page>` и ожидает ввода номера из списка:

```
select page> 3
```

После ввода корректного номера программа запускает браузер (`xdg-open`) на соответствующей странице Википедии и возвращается к основному приглашению:

```
select page> 3
> _
```

Если пользователь введёт `q` вместо номера, выбор будет отменён, и программа вернётся к основному приглашению без открытия браузера.

2.4 Индикатор ожидания и его роль

Индикатор ожидания — отдельная сопрограмма, которая запускается на время сетевого запроса и останавливается, когда приходит ответ. С точки зрения пользователя это означает, что интерфейс остаётся отзывчивым во время долгого соединения: каждая итерация индиктора происходит примерно раз в 100 мс, тогда как основной REPL-цикл в это время заблокирован на ожидании сетевого ответа (но не процессор, системный вызов poll(2) приостанавливает работу программы, пока не появятся интересующие события, не тратя ресурсы).

2.5 Обработка ошибочного ввода

Нечисловой номер страницы. Если в ответ на `select page>` ввести строку, которая не начинается с цифры, преобразование `strtol` вернёт 0, что меньше 1 и поэтому не попадёт в допустимый диапазон:

```
select page> abc
entered number is not in the list
select page> _
```

Номер вне диапазона. Аналогично, число больше количества найденных статей или меньше единицы отклоняется с тем же сообщением.

Отмена выбора. Ввод `q` на приглашении `select page>` трактуется как отказ от открытия страницы. Программа возвращается в главное приглашение.

2.6 Сетевые ошибки

Если на этапе соединения или TLS-рукопожатия произошла ошибка, программа печатает диагностику в стандартный поток ошибок и возвращается к приглашению:

```
> test
Searching... error: connection failed
> _
```

Различаются три класса ошибок:

- `error: connection failed` — не удалось установить TCP-соединение с сервером (DNS не разрешился, сервер недоступен, порт закрыт);
- `error: transport failure` — соединение установлено, но произошла ошибка на уровне TLS или HTTP (обрыв соединения, некорректный ответ сервера);
- `error: invalid response` — сервер вернул HTTP-код, отличный от 200 (например, 4xx или 5xx).

2.7 Модель данных

Результат одного запроса описывается структурой `struct wikipedia_resp`, которая содержит:

- `status` — перечисление `enum wiki_resp_status`, одно из `WRS_OK`, `WRS_CONNECTION_FAILED`, `WRS_INVALID_RESPONSE`, `WRS_TRANSPORT_FAILURE`;
- `payload` — указатель на разобранный JSON-ответ (тип `json_object*`), действителен только при `status == WRS_OK`.

Вызывающий код обязан проверять `status` перед обращением к `payload`. Это часть неявного контракта между `request_wikipedia` и её вызывающим.

3 Примеры работы и описание

Ниже приведены типовые сценарии работы. Строки, начинающиеся с `> `, — это то, что вводит пользователь; остальные строки — вывод программы.

3.1 Успешный поиск и открытие страницы

```
$ ./wikipedia-cli
> корутина
Searching... \
1: Сопрограмма
2: Корутина
3: Асинхронный вводвывод-
4: Fibers
5: Поток выполнения
6: Зелёные потоки
7: Легковесные процессы
8: Concurrency
9: Setcontext
10: Ucontext
select page> 1
> _
```

После выбора страницы браузер открывается на `ru.wikipedia.org/w/index.php?curid=XXXXXXX`. Сообщений об успехе программа не печатает, открытие браузера считается само собой разумеющимся.

3.2 Отмена выбора страницы

```
> haskell
Searching... |
1: Haskell
2: GHC
3: Glasgow Haskell Compiler
...
select page> q
> _
```

Ввод `q` не открывает браузер и не печатает сообщений.

3.3 Ошибка DNS или соединения

Если Wi-Fi отключён и `ru.wikipedia.org` не резолвится:

```
> test
Searching... error: connection failed
> _
```

Программа не завершается и не «зависает», но она возвращает управление в REPL, и следующий запрос можно отправить сразу.

3.4 Обрыв соединения во время запроса

Если соединение было установлено, но сервер закрыл его до завершения ответа:

```
> test
Searching... error: transport failure
> _
```

Этот класс ошибок покрывает также ошибки TLS (например, несовпадение сертификата) и ошибки парсинга HTTP-ответа.

3.5 Сервер вернул 4xx/5xx

Если сервер вернул HTTP-код, отличный от 200:

```
> test
Searching... error: invalid response
> _
```

3.6 Некорректный номер страницы

```
> linux
Searching... /
1: Linux
2: Ядро Linux
3: GNU
...
select page> 42
entered number is not in the list
select page> abc
entered number is not in the list
select page> 2
> _
```

3.7 Завершение через Ctrl+D

```
> quit
$
```

Программа закрывает SSL-контекст, освобождает все выделенные ресурсы и выходит с кодом 0. Слово `quit` здесь не команда; программа получает `EOF` на стандартном вводе, когда пользователь нажимает `Ctrl+D`; строка `quit` в примере — это просто демонстрация того, что при этом происходит.

4 Текст программы

src/io/bufio.h:

```
#ifndef BUFIO_H_
#define BUFIO_H_

#ifndef BUFIODEF
#define BUFIODEF
#endif

#include <stddef.h>

struct bufio {
    void *handler;
    int (*read)(void *, void *, int);
    void (*write)(void *, const void *, int);
    int rb_avail, rb_pos;
    int wb_avail, wb_pos;
    char readbuf[4096];
    char writebuf[4096];
};

BUFIODEF void bio_init(struct bufio *, void *, int (*)(void *, void *, int), void (*)(void
    *, const void *, int));
BUFIODEF void bio_read(struct bufio *, void *, int);
BUFIODEF void bio_send(struct bufio *, const void *, int);
BUFIODEF int  bio_pending(struct bufio *);
BUFIODEF void bio_printf(struct bufio *, const char *, ...);
BUFIODEF int  bio_printf_r(struct bufio *, char *, size_t, const char *, ...);
BUFIODEF void bio_flush(struct bufio *);
BUFIODEF int  bio_read_until(struct bufio *, void *, const char*, size_t);

#endif /* BUFIO_H_ */
```

src/spinner.h:

```
#ifndef SPINNER_H_
#define SPINNER_H_

#include "event_loop2.h"

ev_task *spinner_start(const char *message);
void spinner_stop(ev_task *sp);

#endif
```

src/strutils.h:

```
#ifndef STRUTILS_H_
#define STRUTILS_H_

char *urlencode(const char *p);
void strip(char **line);

#endif /* STRUTILS_H_ */
```

src/wikipedia.h:

```
#ifndef WIKIPEDIA_H_
#define WIKIPEDIA_H_

#include <json-c/json.h>

#include "eloop_ssl.h"

enum wiki_resp_status {
    WRS_OK,
    WRS_CONNECTION_FAILED,
    WRS_INVALID_RESPONSE,
    WRS_TRANSPORT_FAILURE
};

struct wikipedia_resp {
    enum wiki_resp_status status;
    json_object *payload;
};

void request_wikipedia(struct wikipedia_resp *resp, const char *encoded, essl_ctx_t ssl);

#endif
```

src/http.c:

```
#define ELOOP_HTTP_IMPLEMENTATION
#define ELOOP_SSL_IMPLEMENTATION
#define ELOOP_NET_IMPLEMENTATION
#define EVENT_LOOP2_IMPLEMENTATION
#include "eloop_http.h"
```

src/io/bufio.c:

```
#define _XOPEN_SOURCE 500
#include "bufio.h"

#include <stdio.h>
#include <string.h>
#include <stdarg.h>
#include <assert.h>

static void bio_fetch(struct bufio *bio)
{
    int avail = sizeof bio->readbuf - bio->rb_avail - bio->rb_pos;
    int n = bio->read(bio->handler, &bio->readbuf[bio->rb_pos + bio->rb_avail], avail);
    bio->rb_avail += n;
}

BUFIODEF void bio_read(struct bufio *bio, void *buf, int size)
{
    int pos = 0;
    while (pos < size) {
        int n;
        if (bio->rb_avail == 0) bio_fetch(bio);

        n = size - pos;
```

```
        if (n >= bio->rb_avail) n = bio->rb_avail;
        assert(n >= 0);
        memcpy((char*)buf + pos, &bio->readbuf[bio->rb_pos], n);
        pos += n;
        bio->rb_pos = (bio->rb_pos + n) % sizeof bio->readbuf;
        bio->rb_avail -= n;
    }
}

BUFIODEF void bio_send(struct bufio *bio, const void *buf, int size)
{
    int pos = 0;
    while (pos < size) {
        int n, free;
        if (bio->wb_pos + bio->wb_avail == sizeof bio->writebuf)
            bio_flush(bio);

        n = size - pos;
        free = (int) sizeof bio->writebuf - bio->wb_pos - bio->wb_avail;
        if (n >= free) n = free;
        assert(n >= 0);
        memcpy(&bio->writebuf[bio->wb_pos + bio->wb_avail], (char*)buf + pos, n);
        pos += n;
        bio->wb_avail += n;
    }
}

BUFIODEF void bio_flush(struct bufio *bio)
{
    bio->write(bio->handler, &bio->writebuf[bio->wb_pos], bio->wb_avail);
    bio->wb_pos = (bio->wb_pos + bio->wb_avail) % sizeof bio->writebuf;
    bio->wb_avail = 0;
}

BUFIODEF void bio_init(struct bufio *bio, void *hnd,
        int (*read)(void *, void *, int),
        void (*write)(void *, const void *, int))
{
    memset(bio, 0, sizeof(*bio));
    bio->handler = hnd;
    bio->read = read;
    bio->write = write;
}

static int buf_ends_with(const char *buf, size_t size,
        const char *pat, size_t pat_len)
{
    if (size < pat_len) return 0;
    if (memcmp(&buf[size - pat_len], pat, pat_len) != 0) return 0;
    return 1;
}

BUFIODEF int bio_read_until(struct bufio *bio, void *_buf,
        const char *pattern, size_t limit)
{
    char *buf = _buf;
    size_t size = 0, pat_len = strlen(pattern);

```

```
    while (size < limit && !buf_ends_with(buf, size, pattern, pat_len))
        bio_read(bio, &buf[size++], 1);

    if (size < limit) return size;
    return -1;
}

static int bio_printf_rv(struct bufio *bio, char *buf, size_t n, const char *fmt, va_list
    va)
{
    int ret = vsnprintf(buf, n, fmt, va);
    if (ret < 0 || (size_t) ret >= n) return -1;
    bio_send(bio, buf, n);
    return ret;
}

BUFIODEF int bio_printf_r(struct bufio *bio, char *buf, size_t n, const char *fmt, ...)
{
    va_list va;
    int ret;

    va_start(va, fmt);
    ret = bio_printf_rv(bio, buf, n, fmt, va);
    va_end(va);
    return ret;
}

BUFIODEF void bio_printf(struct bufio *bio, const char *fmt, ...)
{
    static char buf[4096];
    va_list va;
    int ret;

    va_start(va, fmt);
    ret = bio_printf_rv(bio, buf, sizeof buf, fmt, va);
    assert(ret >= 0);
    va_end(va);
}
```

src/main.c:

```
#include "eloop_ssl.h"

#include <unistd.h>

void userloop(void);

static int64_t stdin_cb(void *d)
{
    (void) d;
    return STDIN_FILENO;
}

static void run(void) {
    ev_set_data_to_transport_handle_cb(stdin_cb);
    ev_set_flags(EV_F_READ);
    ev_set_timeout(-1);
    for (;;) userloop();
```

```
}

int main(void)
{
    event_loop loop;
    essl_ctx_t ssl;
    ev_task *t;

    eloop_init(&loop);
    ssl = essl_init();

    t = eloop_add(&loop, run);
    ev_set_task_data(t, ssl);

    eloop_run(&loop);
    essl_deinit(ssl);
    return 0;
}
```

src/spinner.c:

```
#include "spinner.h"

#include <stdio.h>
#include <unistd.h>

static void spinner(void)
{
    int i;
    ev_set_timeout(100);

    printf(" ");
    for (i = 0;;) {
        printf("\b%c", "-\\|/"[i++%4]);
        fflush(stdout);
        if (ev_recv_message()) break;
    }
    printf("\r\x1b[K");
}

ev_task *spinner_start(const char *message)
{
    event_loop *loop = ev_get_loop();
    printf("%s", message);
    return eloop_add(loop, spinner);
}

void spinner_stop(ev_task *sp)
{
    ev_send_message(sp, (void *)1);
}
```

src/strutils.c:

```
#include "strutils.h"

#include <stdlib.h>
#include <string.h>
```

```c
#include <ctype.h>

char *urlencode(const char *p)
{
    char *e = malloc(strlen(p)*3 + 1);
    char *c = e;
    for (; *p; ++p) {
        unsigned char q = *p;
        if (isalnum(q)) {
            *c++ = q;
            continue;
        }
        *c++ = '%';
        *c++ = (q>>4) < 10 ? (q>>4) + '0' : (q>>4) + 'A' - 10;
        *c++ = (q&0xf) < 10 ? (q&0xf) + '0' : (q&0xf) + 'A' - 10;
    }
    *c = 0;
    return e;
}

void strip(char **line)
{
    int len;
    while (isspace(**line)) (*line)++;
    len = strlen(*line);
    while (len && isspace((*line)[len-1])) (*line)[--len] = 0;
}
```

src/userloop.c:

```c
#define _XOPEN_SOURCE 500
#include <stdio.h>

#include "strutils.h"
#include "wikipedia.h"
#include "spinner.h"

static char *prompt(const char *prompt, char *buf, size_t len)
{
    char *line;
    printf("%s> ", prompt);
    fflush(stdout);
    if (ev_yield() & EV_F_EXCEPTIONS) return NULL;
    line = fgets(buf, len, stdin);
    if (!line) return NULL;
    strip(&line);
    return line;
}

static json_object *search(const char *line)
{
    struct wikipedia_resp resp;
    ev_task *spinner;
    essl_ctx_t ssl = ev_get_data();
    char *encoded = urlencode(line);
    spinner = spinner_start("Searching... ");
    request_wikipedia(&resp, encoded, ssl);
    spinner_stop(spinner);
```

```
    free(encoded);

    switch (resp.status) {
    case WRS_OK: return resp.payload;
    case WRS_CONNECTION_FAILED: fprintf(stderr, "error: connection failed\n"); break;
    case WRS_INVALID_RESPONSE: fprintf(stderr, "error: invalid response\n"); break;
    case WRS_TRANSPORT_FAILURE: fprintf(stderr, "error: transport failure\n"); break;
    }
    return NULL;
}

static json_object *select_variant(json_object *payload)
{
    size_t i;
    const char *line;
    json_object *walk;
    char buffer[256];
    walk = payload;
    if (!json_object_object_get_ex(walk, "query", &walk)) return NULL;
    if (!json_object_object_get_ex(walk, "search", &walk)) return NULL;

    for (;;) {
        line = prompt("select page", buffer, sizeof buffer);
        if (!line) return NULL;

        if (*line == 'q') return NULL;
        i = strtol(line, NULL, 10);
        i -= 1;
        if (i < json_object_array_length(walk)) break;
        printf("entered number is not in the list\n");
    }

    walk = json_object_array_get_idx(walk, i);
    if (!json_object_object_get_ex(walk, "pageid", &walk)) {
        fprintf(stderr, "error: wikipedia api changed\n");
        return NULL;
    }
    if (json_object_get_type(walk) != json_type_int) {
        fprintf(stderr, "error: wikipedia api changed\n");
        return NULL;
    }
    return walk;
}

static int display_variants(json_object *payload)
{
    size_t i;
    json_object *walk;
    walk = payload;
    if (!json_object_object_get_ex(walk, "query", &walk)) return 0;
    if (!json_object_object_get_ex(walk, "search", &walk)) return 0;

    for (i = 0; i < json_object_array_length(walk); ++i) {
        json_object *title;
        json_object *result = json_object_array_get_idx(walk, i);
        if (!json_object_object_get_ex(result, "title", &title)) continue;
        if (json_object_get_type(title) != json_type_string) continue;
        printf("%ld: %s\n", i + 1, json_object_get_string(title));
```

```
    }
    return 1;
}

static void open_in_browser(json_object *obj)
{
    char buffer[256];
    snprintf(buffer, sizeof buffer, "xdg-open https://ru.wikipedia.org/w/index.php?curid=%d
    2>/dev/null", json_object_get_int(obj));
    system(buffer);
}

void userloop(void)
{
    json_object *selected, *payload;
    char *line;
    char buffer[256];

    line = prompt("", buffer, sizeof buffer);
    if (!line) ev_terminate();

    payload = search(line);
    if (!payload) return;
    if (!display_variants(payload)) goto cleanup;
    selected = select_variant(payload);
    if (!selected) goto cleanup;
    open_in_browser(selected);

 cleanup:
    json_object_put(payload);
}
```

src/wikipedia.c:

```
#define _XOPEN_SOURCE 500

#include "wikipedia.h"
#include "eloop_http.h"

#include <stdio.h>

#define HOST "ru.wikipedia.org"
#define PORT 443

struct wikipedia_req_closure {
    struct wikipedia_resp *resp;
    const char *encoded_query;
    ev_task *parent;
    essl_ctx_t ssl;
};

static void wikipedia_request(enet_socket *sock)
{
    struct wikipedia_req_closure *reqc;
    struct http_resp resp;
    jmp_buf err;
    http_transport http;
    json_object *jobj = NULL;
```

```
    json_tokener *volatile jtk = NULL;
    char *volatile buf = NULL;
    size_t read;
    int len;
    reqc = esock_get_data(sock);

    if (https_connect(&http, reqc->ssl, sock, &err) < 0) {
        reqc->resp->status = WRS_CONNECTION_FAILED;
        goto fini;
    }

    switch (setjmp(err)) {
    case HTTP_ERR_NONE: break;
    case HTTP_ERR_TRANSPORT:
    case HTTP_ERR_PARSE:
    case HTTP_ERR_PARSE_LIMIT:
        reqc->resp->status = WRS_TRANSPORT_FAILURE;
        goto fini;
    }

    len = snprintf(NULL, 0,
    "/w/api.php?action=query&list=search&utf8=&format=json&srsearch=%%22%s%%22",
    reqc->encoded_query);
    buf = malloc(len + 1);
    snprintf(buf, len + 1,
    "/w/api.php?action=query&list=search&utf8=&format=json&srsearch=%%22%s%%22",
    reqc->encoded_query);
    http_send_req(&http, "GET", buf);
    free(buf);
    buf = NULL;
    http_send_header(&http, "Host", HOST);
    http_send_header(&http, "User-Agent", "curl/8.0");
    http_send_header(&http, "Connection", "closed");
    http_end_request(&http);

    http_recv_resp(&http, &resp, NULL);

    if (resp.status_code != 200) {
        reqc->resp->status = WRS_INVALID_RESPONSE;
        goto fini;
    }

    read = 0;
    if (resp.content_length > 0) {
        len = 4096;
        buf = malloc(len);
    }
    jtk = json_tokener_new();
    do {
        char chunk[16] = {0};
        if (resp.content_length > 0) {
            if ((size_t)resp.content_length < read + len)
                len = resp.content_length - read;
        } else {
            /* this is hackish, thank you http, very cool */
            unsigned short term = '\r'|('\n'<<8);
            char *c = chunk;
            do {
```

```
                http_recv(&http, c++, 1);
                if ((size_t)(c - chunk) > sizeof chunk) {
                    reqc->resp->status = WRS_TRANSPORT_FAILURE;
                    goto fini;
                }
            } while (c - chunk < 2 || ((unsigned short*)c)[-1] != term);
            len = strtol(chunk, NULL, 16);
            free(buf);
            buf = malloc(len);
        }
        http_recv(&http, buf, len);
        read += len;
        if (resp.content_length < 0) http_recv(&http, chunk, 2); /* extra \r\n */
        jobj = json_tokener_parse_ex(jtk, buf, len);
    } while (!jobj);

    reqc->resp->status = WRS_OK;
    reqc->resp->payload = jobj;

 fini:
    free(buf);
    if (jtk) json_tokener_free(jtk);
    ev_send_message(reqc->parent, NULL);
}

void request_wikipedia(struct wikipedia_resp *resp, const char *encoded, essl_ctx_t ssl)
{
    event_loop *loop = ev_get_loop();
    /* since this request function is blocking for the caller, the
       request closure can be safely located on the current stack
       frame */
    struct wikipedia_req_closure reqc;

    reqc.resp = resp;
    reqc.encoded_query = encoded;
    reqc.parent = ev_current();
    reqc.ssl = ssl;

    enet_connect_tcp(loop, HOST, PORT, wikipedia_request, &reqc);
    ev_recv_message();
}
```

5 Выводы

В ходе выполнения практической работы было разработано консольное приложение — клиент поиска по Википедии — на языке C с использованием собственной библиотеки асинхронного ввода-вывода. Программа выполняет HTTPS-запросы, парсит JSON, отображает результаты и открывает выбранную статью в браузере, не блокируя пользовательский интерфейс.

Реализованная функциональность

В соответствии с заданием были реализованы:

- интерактивный REPL-цикл с обработкой `Ctrl+D` (EOF) как команды завершения;
- асинхронное HTTPS-соединение с `ru.wikipedia.org`, не блокирующее основной цикл приложения;
- парсинг HTTP-ответа (заголовки, тело, chunked encoding) и парсинг JSON-ответа через библиотеку `json-c`;
- индикатор прогресса в виде отдельной сопрограммы, обновляющийся каждые 100 мс;
- URL-кодирование поискового запроса;
- обработка ошибок трёх классов: сетевые, транспортные, ошибки валидации ответа;
- открытие браузера на выбранной статье через `xdg-open`.

Соблюдение принципов объектно-ориентированного программирования

1. **Инкапсуляция.** Все структуры данных (`struct wikipedia_resp`, `struct enet_socket`, `struct bufio`, `struct http_transport`) объявлены как непрозрачные типы в заголовочных файлах; их поля видны только внутри реализующего модуля. Изменение состояния выполняется исключительно через функции-методы (`http_send_req`, `esock_recv`, `bio_flush` и т. д.), которые проверяют предусловия и возвращают коды ошибок.
2. **Композиция.** Приложение построено снизу вверх: `event_loop` управляет сопрограммами; `eloop_net` обеспечивает сокеты; `eloop_ssl` оборачивает их в TLS; `eloop_http` строит HTTP-клиент над транспортом через `bufio`; `wikipedia` инкапсулирует бизнес-логику запроса; `userloop` управляет интерфейсом. Каждый слой владеет своими подобъектами и освобождает их при завершении.
3. **Полиморфизм.** В коде активно используются указатели на функции как способ абстракции над поведением:
    - `bio_init` принимает указатели на `read` и `write`, что позволяет HTTP-клиенту одинаково работать поверх TCP и поверх TLS без изменения своего кода;

- `esock_setup_transport` принимает указатель на функцию очистки, что позволяет подключать произвольные транспорты (TLS, SOCKS5-прокси) к одному сокету;
- `esock_callbacks` и `essl_callbacks` реализуют паттерн «наблюдатель» для событий закрытия, ошибок и таймаутов;
- `eloop_add` принимает `ev_routine` — функцию без аргументов, — что позволяет запускать произвольный код как сопрограмму.

Во всех случаях выбор конкретной реализации происходит в момент создания объекта, а не в момент вызова — классический паттерн «стратегия».

4. **Абстракция.** Вызывающий код (`userloop.c`) не знает, что находится под `request_wikipedia`: сокеты, TLS, HTTP, JSON — всё скрыто за одной функцией, которая принимает строку и возвращает структуру с результатом. Аналогично, HTTP-клиент не знает, работает он поверх TCP или TLS, — для него есть только `read`/`write` callbacks. Каждый слой скрывает детали нижнего слоя за минимальным интерфейсом.

5. **Разделение ответственности.** Три уровня (транспорт, клиент API, интерфейс пользователя) ничего не знают друг о друге, кроме непосредственного соседа снизу. Изменение, например, формата вывода результатов не затрагивает ни HTTP-клиент, ни сетевой слой; добавление нового API-эндпоинта требует изменения только `wikipedia.c`.

Заключение

Поставленная задача выполнена в полном объёме. Программа функционирует согласно требованиям задания, поддерживает интерактивный интерфейс, корректно обрабатывает сетевые и пользовательские ошибки, демонстрирует применение принципов объектно-ориентированного проектирования в проекте на языке C.

Полученный код легко расширяется: добавление нового API-эндпоинта сводится к написанию новой функции в `wikipedia.c`; замена `xdg-open` на платформо-специфичный запуск браузера — к правке одной функции в `userloop.c`; замена формата вывода результатов не затрагивает ни HTTP-клиент, ни сетевой слой.